

Major Project- II

Software Requirements Specification

For

OpsPilot – An AI-Driven DevSecOps Pipeline Automation Platform

13th March 2026

Prepared by

Specialization	SAP ID	Name
B.tech CSE, DevOps	500107141	Aayush Mishra
B.tech CSE, DevOps	500105056	Rajat Kamboj
B.tech CSE, DevOps	500102126	Raghav Kamboj
B.tech CSE, DevOps	500107193	Piyush Kumar



Department of Informatics
School Of Computer Science
UNIVERSITY OF PETROLEUM & ENERGY STUDIES,
DEHRADUN- 248007. Uttarakhand

Under the guidance of
Dr. Hitesh Kumar Sharma

Table of Contents

Topic		Page No
1	Introduction	3
	1.1 Purpose of the Project	3
	1.2 Target Beneficiary	4
	1.3 Project Scope	4
2	Project Description	5
	2.1 Reference Algorithm	5
	2.2 Data/ Data structure	6
	2.3 SWOT Analysis	6
	2.4 Project Features	6
	2.5 User Classes and Characteristics	7
	2.6 Design and Implementation Constraints	7
	2.7 Design diagrams	7
	2.8 Assumption and Dependencies	7
3	System Requirements	7
	3.1 User Interface	8
	3.2 Software Interface	8
	3.3 Database Interface	8
	3.4 Protocols	9
	3.5 Functional Requirements	9
	3.6 External Interface Requirements	11
	3.7 Use Case Description	12
	3.8 System Architecture	13
	3.9 Risk Management	14
	3.10 Acceptance Criteria	15
	3.11 DFD(Data Flow Digram)/ Class Diagram/ Sequence Diagram	15
4	Non-functional Requirements	17
	4.1 Performance requirements	17

	4.2 Security requirements	17
	4.3 Software Quality Attributes	18
5	Other Requirements	18
6	References	18

1. Introduction

The rapid growth of cloud computing, microservices, and software-as-a-service (SaaS) has drastically changed how applications are built and delivered. Modern organizations rely heavily on Continuous Integration and Continuous Deployment (CI/CD) pipelines to ensure fast, reliable, and secure delivery of software. DevOps tools such as Jenkins, GitHub Actions, Docker, Kubernetes, Terraform, SonarQube, and cloud platforms like AWS have become essential components of modern software engineering.

However, these tools are highly complex and require engineers to write large configuration files, manage scripts, handle secrets, integrate security tools, and monitor production systems. Even a simple deployment requires knowledge of multiple technologies and scripting languages. This creates a steep learning curve for students and junior developers and increases operational costs for companies.

OpsPilot is designed to solve this problem by introducing **AI-driven, DevOps automation**. In this system, users interact with the CI/CD platform using natural language commands instead of scripts. The system uses artificial intelligence to understand the intent of the user, automatically generate pipelines, run DevSecOps tools, and deploy applications securely to the cloud.

This project brings together Artificial Intelligence, DevOps, Cloud Computing, and Cybersecurity into a single intelligent automation platform.

1.1 Purpose of the Project

The purpose of the **OpsPilot** project is to create an intelligent, AI-driven DevSecOps automation platform that can perform most of the responsibilities traditionally handled by a DevOps team. In modern software organizations, DevOps engineers are responsible for building CI/CD pipelines, configuring cloud infrastructure, managing Docker containers, running security scans, deploying applications, and monitoring production systems. These activities require high technical expertise, consume significant time, and increase operational costs for organizations.

OpsPilot is designed to replace or significantly reduce the dependency on manual DevOps teams by automating the entire pipeline using Artificial Intelligence and natural language commands. Instead of having a team of engineers writing Jenkins files, Kubernetes configurations, Terraform scripts, and security policies, OpsPilot allows a single user to control all these operations through simple prompts such as *“Deploy my application securely to AWS”* or *“Scan my code and fix vulnerabilities.”*

The system uses AI to understand user intent, automatically design the CI/CD pipeline, select appropriate tools, configure cloud resources, and execute all required steps without human intervention. This means that even a non-expert user can deploy enterprise-grade, secure, and scalable applications. By doing so, OpsPilot significantly reduces manpower, eliminates human errors, and speeds up the software delivery lifecycle.

Another important purpose of the project is to enable startups, educational institutions, and small to medium enterprises to run production-level DevOps operations without hiring large and expensive DevOps teams. This is especially valuable for startups that need to scale quickly but cannot afford dedicated DevOps staff. At the same time, professional DevOps engineers can use OpsPilot to move away from repetitive manual tasks and focus on high-level system design, optimization, and security strategy.

In summary, **OpsPilot** is built to act as an autonomous DevSecOps engineer — a smart system that can build, secure, deploy, and monitor software applications automatically using human-language instructions.

1.2 Target Beneficiary

The OpsPilot platform is designed to serve a wide range of users who are involved in modern software development and deployment. The primary beneficiaries of this system are software developers, DevOps engineers, startups, educational institutions, and small to medium-scale enterprises.

For software developers, OpsPilot significantly reduces the effort required to deploy applications. Developers often struggle with configuring pipelines, managing cloud resources, and integrating security checks. With OpsPilot, they can focus on writing code while the system handles the deployment automatically based on natural language commands.

DevOps engineers benefit from OpsPilot because it eliminates repetitive configuration tasks. Instead of maintaining multiple Jenkins pipelines and YAML files, they can use AI-generated pipelines that adapt dynamically to project needs. This improves productivity and reduces human error.

Startups and small organizations usually do not have dedicated DevOps teams. OpsPilot allows them to deploy applications securely without hiring expensive specialists. Educational institutions can use OpsPilot to teach DevOps concepts in a simplified way, allowing students to understand the process without being overwhelmed by technical complexity.

1.3 Project Scope

The scope of the OpsPilot project covers the complete automation of the DevSecOps lifecycle using artificial intelligence. The system is designed to automate all major stages involved in modern software delivery.

The project includes the following key components:

- Prompt-based interaction for pipeline control
- AI-based interpretation of user commands
- Automatic CI/CD pipeline generation

- Integration with Git repositories
- Containerization using Docker
- Security scanning using industry tools
- Cloud deployment using AWS
- Monitoring and reporting through a dashboard

OpsPilot is not limited to one type of application or programming language. It is designed to support web applications, backend services, and microservices-based architectures. The scope also includes future extensibility for Kubernetes, Azure, and Google Cloud.

2. Project Description

OpsPilot is an intelligent DevSecOps automation platform that allows users to manage software pipelines using natural language commands. Instead of writing scripts or configuration files, users interact with the system through a prompt box. The AI engine understands the request, creates a suitable pipeline, and executes it.

The system acts as a bridge between the user and complex DevOps tools. It hides technical complexity and provides an easy-to-use interface for managing application lifecycle operations.

2.1 Reference Algorithm

The reference algorithm used in OpsPilot is based on **Intent Recognition and Workflow Orchestration**. The algorithm works in multiple stages to convert a user's natural language prompt into a working DevOps pipeline.

The steps are as follows:

1. The user enters a prompt describing the required action.
2. The AI engine processes the prompt using Natural Language Processing techniques.
3. The system extracts key information such as application type, cloud platform, security requirements, and pipeline stages.
4. This information is converted into a structured pipeline definition in JSON format.
5. The pipeline generator converts this data into executable scripts.
6. The execution engine runs the pipeline step by step.
7. Logs and results are stored and displayed to the user.

This algorithm allows dynamic and flexible pipeline generation without manual configuration.

2.2 Data and Data Structures

OpsPilot uses structured and unstructured data to operate efficiently. The main types of data include user prompts, pipeline configurations, execution logs, security scan results, and deployment information.

All these are stored using JSON-based data structures. JSON is chosen because it is flexible, lightweight, and easy to parse. Each pipeline is represented as a structured JSON object containing stages, tools, environment variables, and execution commands.

MongoDB is used as the database because it can store dynamic and nested JSON documents efficiently.

2.3 SWOT Analysis

A SWOT analysis is performed to evaluate the feasibility of OpsPilot.

Strengths

OpsPilot uses AI to automate DevOps, which makes it easy to use, fast, and highly innovative. It reduces manual errors and increases productivity.

Weaknesses

The system depends on internet connectivity, cloud services, and AI APIs. Any downtime in these services can affect operation.

Opportunities

The global DevOps and AI markets are rapidly growing. OpsPilot can be used by startups, enterprises, and educational institutions.

Threats

Security risks, data privacy issues, and competition from existing DevOps platforms pose potential threats.

2.4 Project Features

OpsPilot includes a wide range of features that make it a complete DevSecOps automation platform. The system allows users to create pipelines using natural language. It supports automatic build, test, containerization, security scanning, and deployment. It also provides real-time logs, error reporting, and performance monitoring.

The platform also supports role-based access, project history, and pipeline reuse.

2.5 User Classes and Characteristics

The system supports three major user types.

Developer:

Uses the system to build and deploy applications quickly.

DevOps-Engineer:

Monitors pipelines, manages security and cloud infrastructure.

Administrator:

Controls system settings, user permissions, and resource usage.

Each user has different access levels and responsibilities.

2.6 Design and Implementation Constraints

OpsPilot must work within certain constraints. It requires cloud credentials, AI model availability, network connectivity, and sufficient computing resources. The system must also comply with security and privacy regulations.

2.7 Design Diagrams

The project includes several UML diagrams such as use case diagram, sequence diagram, data flow diagram, and system architecture diagram. These diagrams explain how users interact with the system and how data flows between components.

2.8 Assumptions and Dependencies

The system assumes that users provide valid Git repositories and cloud credentials. It also depends on external services such as AI APIs, Docker, AWS, and security scanning tools.

3. System Requirements

The OpsPilot system requires a well-defined set of user, software, database, and communication interfaces to ensure smooth and reliable operation. Since the platform connects AI models, DevOps tools, cloud services, and users, it must support seamless interaction between all components. This section describes how

users interact with the system, how different software components communicate, how data is stored and retrieved, and how information flows across the network.

3.1 User Interface

The user interface of OpsPilot is designed to be simple, intuitive, and powerful. The main interface is a web-based dashboard that contains a command prompt box where users type their natural language instructions. This prompt box acts as the primary interaction point between the user and the system.

Apart from the command box, the dashboard displays the status of pipelines, execution logs, security scan reports, and deployment URLs. Users can see whether a build is running, successful, or failed. Color-coded indicators are used to make it easy to understand the status of each stage. The interface also allows users to view previous commands, reuse pipelines, and track project history.

The interface is designed so that even a beginner can deploy an application without understanding complex DevOps configurations. Advanced users such as DevOps engineers can view detailed logs, security findings, and performance metrics.

3.2 Software Interface

OpsPilot interacts with a wide range of external software tools to perform DevOps operations. These include version control systems such as GitHub and GitLab, container platforms such as Docker, cloud platforms such as AWS, and security tools such as SonarQube and Trivy.

The backend of OpsPilot communicates with these tools using APIs, command-line interfaces, and automation scripts. For example, GitHub APIs are used to fetch code repositories, Docker CLI is used to build container images, AWS CLI is used to create cloud resources, and security tools are invoked to scan code and images.

The AI model is integrated using REST APIs. The user's prompt is sent to the AI service, which returns a structured interpretation of the request. This interpretation is then passed to the pipeline generation module.

3.3 Database Interface

The database is a critical component of OpsPilot because it stores all operational data. MongoDB is used as the database because it supports flexible, JSON-based data storage, which is ideal for storing pipeline definitions and logs.

The database stores user profiles, project details, prompts, generated pipelines, execution logs, security reports, and deployment metadata. Every pipeline run is recorded so that users can review previous deployments, troubleshoot errors, and audit security results.

The backend uses secure database drivers to read and write data. Role-based access control ensures that users can only access their own projects and logs.

3.4 Protocols

OpsPilot uses secure and standard communication protocols to ensure data integrity and privacy. All communication between the user interface and backend occurs over HTTPS to protect sensitive data such as API keys and cloud credentials.

The backend communicates with AI services and DevOps tools using RESTful APIs. For real-time updates such as live logs and build status, WebSocket connections are used. This allows users to see pipeline progress instantly without refreshing the page.

Authentication and authorization are handled using secure token-based protocols such as JWT.

3.5 Functional Requirements

Functional requirements describe the core operations that the OpsPilot system must perform.

FR1 – User Authentication

The system shall allow users to register and log in using secure authentication mechanisms. Only authenticated users will be allowed to access the platform.

FR2 – Project Creation

Users shall be able to create new DevOps projects by providing repository links and required configuration details.

FR3 – Natural Language Prompt Processing

The system shall accept natural language prompts from the user and interpret them using AI-based Natural Language Processing.

Example prompts include:

- “Deploy my application to AWS.”
- “Run security scans on my repository.”
- “Build a Docker image and push it.”

FR4 – Automatic Pipeline Generation

Based on the interpreted prompt, the system shall automatically generate CI/CD pipelines.

The pipeline may include stages such as:

- Build
- Test
- Security scanning
- Containerization
- Deployment

FR5 – Integration with Git Repositories

The system shall integrate with version control platforms such as GitHub or GitLab to fetch source code and monitor changes.

FR6 – Containerization

The system shall automatically create Docker images from application code and push them to a container registry.

FR7 – Security Scanning

The system shall run automated security scans including:

- Static code analysis (SAST)
- Dependency scanning (SCA)
- Container image scanning

FR8 – Cloud Deployment

The system shall deploy applications to cloud platforms such as AWS using automated infrastructure scripts.

FR9 – Monitoring and Logging

The system shall provide real-time logs, monitoring data, and execution reports for each pipeline run.

FR10 – Role-Based Access Control

Different users shall have different permission levels such as:

- Developer
- DevOps Engineer
- Administrator

3.6 External Interface Requirements

External interfaces describe how the system interacts with external components.

1. User Interface

OpsPilot will provide a web-based dashboard accessible through modern browsers.

Main UI components include:

- Command prompt interface
- Pipeline status dashboard
- Deployment logs
- Security reports
- Project management panel

2. Hardware Interface

The system will run on standard cloud or server infrastructure.

Minimum requirements include:

- CPU: 4 cores
- RAM: 8 GB
- Storage: 50 GB
- Internet connectivity

3. Software Interface

OpsPilot interacts with multiple software tools including:

Tool	Purpose
GitHub/ GitLab	Source Code repository
Docker	Container creation
AWS	Cloud deployment
SonarQube	Static code analysis
Trivy	Container vulnerability
Prometheus	Monitoring
Grafana	Visualization

4. Communication Interface

The system will use:

- HTTPS for secure communication
- REST APIs for tool integration
- WebSockets for real-time pipeline updates

3.7 Use Case Description

Use Case 1 – Deploy Application

Actor: Developer

Steps:

1. User logs into the OpsPilot dashboard.
2. User enters a prompt such as “Deploy my Node.js application to AWS.”
3. AI interprets the request.
4. System generates a CI/CD pipeline.
5. Pipeline runs build, test, and security scans.
6. Application is deployed to the cloud.
7. Logs and deployment URL are displayed.

Use Case 2 – Run Security Scan

Actor: DevOps Engineer

Steps:

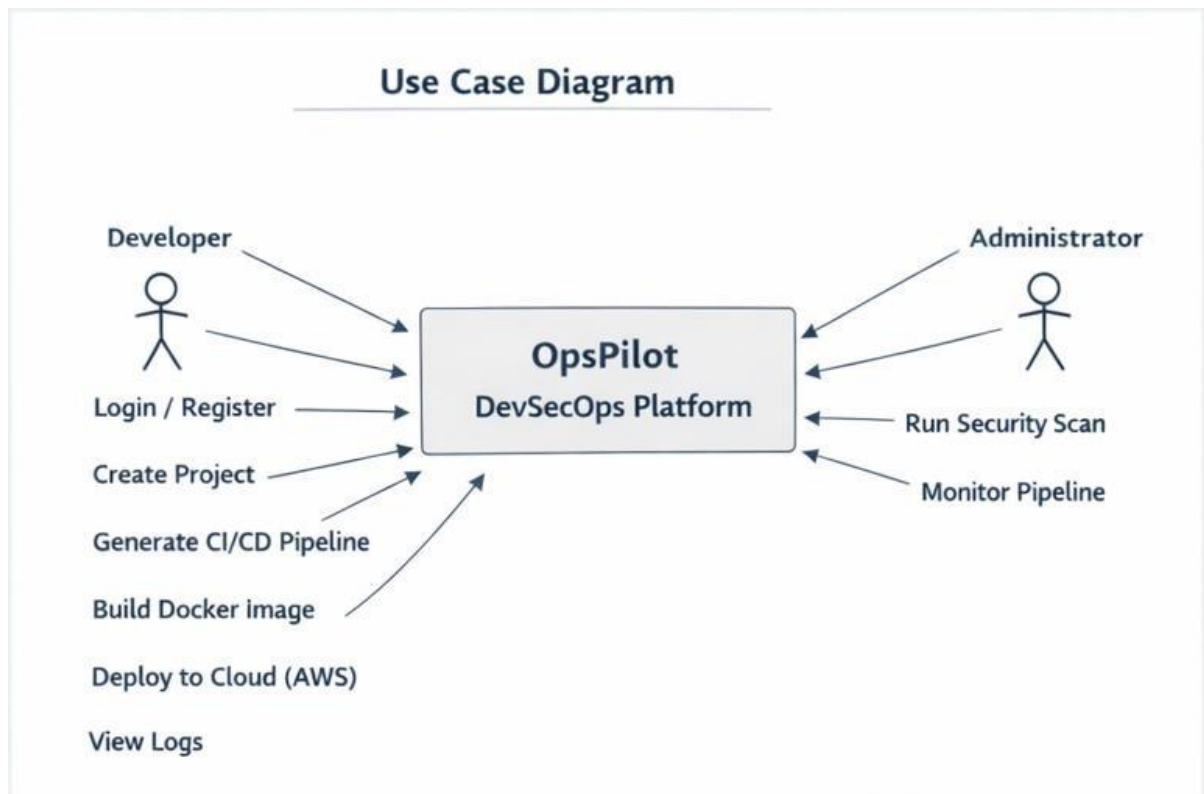
1. User selects an existing project.
2. User requests a security scan.
3. System runs static analysis and container scanning.
4. Vulnerabilities are detected.
5. A security report is generated.

Use Case 3 – Monitor Pipeline

Actor: Administrator

Steps:

1. Admin opens the monitoring dashboard.
2. System displays running pipelines.
3. Admin reviews logs and performance metrics.
4. Alerts are generated if failures occur.



3.8 System Architecture

OpsPilot follows a modular microservice architecture consisting of the following components:

1. User Interface Layer

A web-based dashboard where users interact with the system using natural language prompts.

2. AI Interpretation Engine

This module processes user prompts using Natural Language Processing and converts them into structured pipeline definitions.

3. Pipeline Generation Engine

Generates CI/CD pipelines automatically based on interpreted user commands.

4. Execution Engine

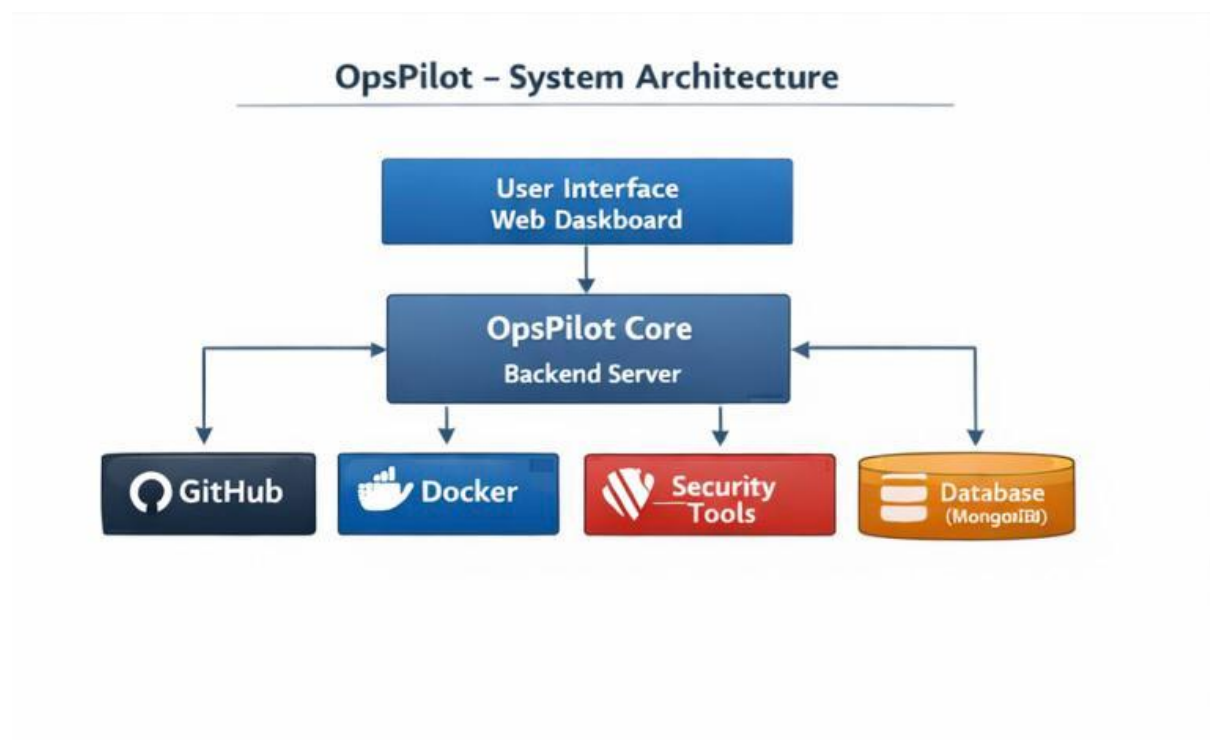
Runs the generated pipeline stages including build, test, security scan, and deployment.

5. Integration Layer

Handles communication with external tools such as GitHub, Docker, AWS, and security scanners.

6. Database Layer

Stores project information, pipeline configurations, execution logs, and security reports using MongoDB.



3.9 Risk Management

The project may face several potential risks.

Technical Risks:

AI interpretation errors may generate incorrect pipelines.

Security Risks:

Sensitive data such as cloud credentials must be protected from unauthorized access.

Infrastructure Risks:

Cloud outages or API failures may interrupt deployments.

Mitigation Strategies

- Implement input validation.
- Use encrypted credential storage.
- Maintain backup systems.
- Implement detailed logging and monitoring.

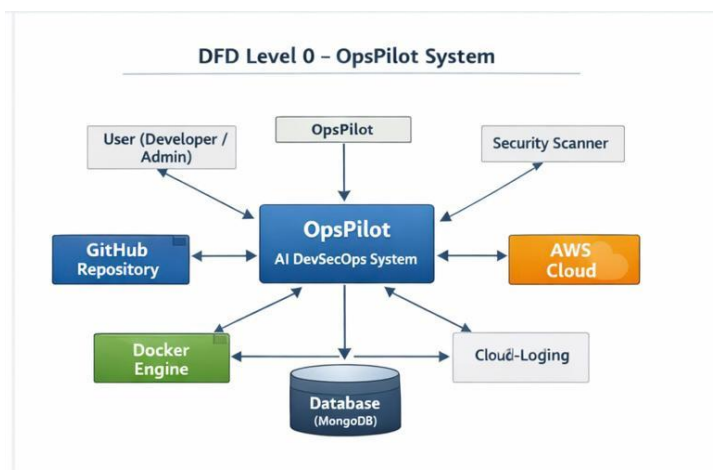
3.10 Acceptance Criteria

The system will be considered successful if the following conditions are met:

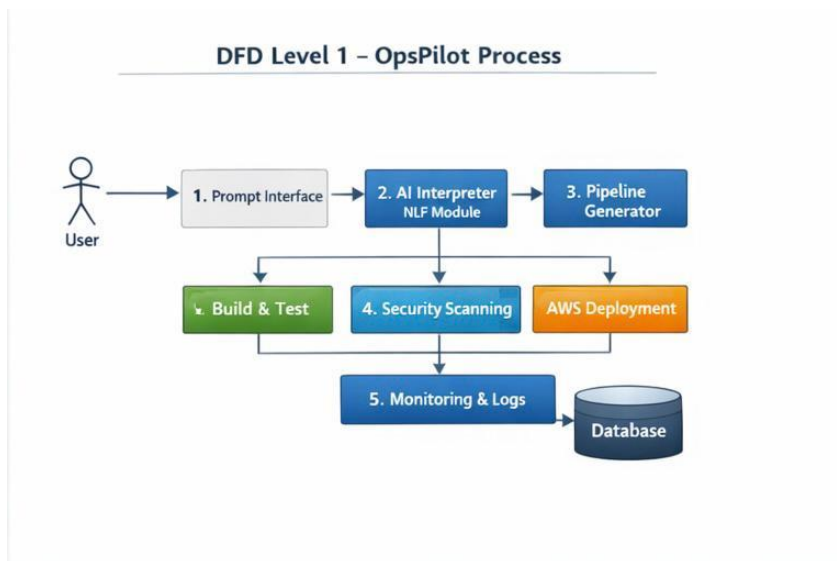
- Users can create and manage DevOps pipelines using natural language prompts.
- The system successfully integrates with Git repositories.
- Security scans run automatically during pipelines.
- Applications can be deployed automatically to AWS.
- The dashboard displays real-time logs and monitoring information.

3.11 DFD(Data Flow Digram)/ Class Diagram/ Sequence Diagram

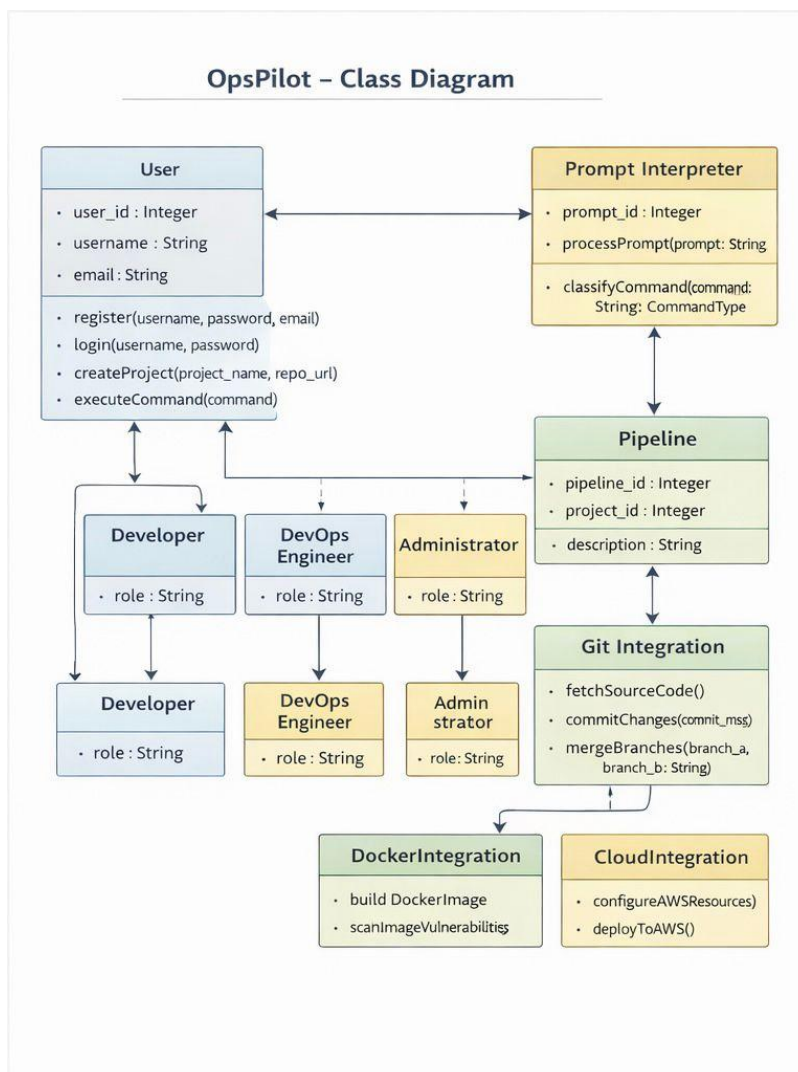
1. DFD level 0:



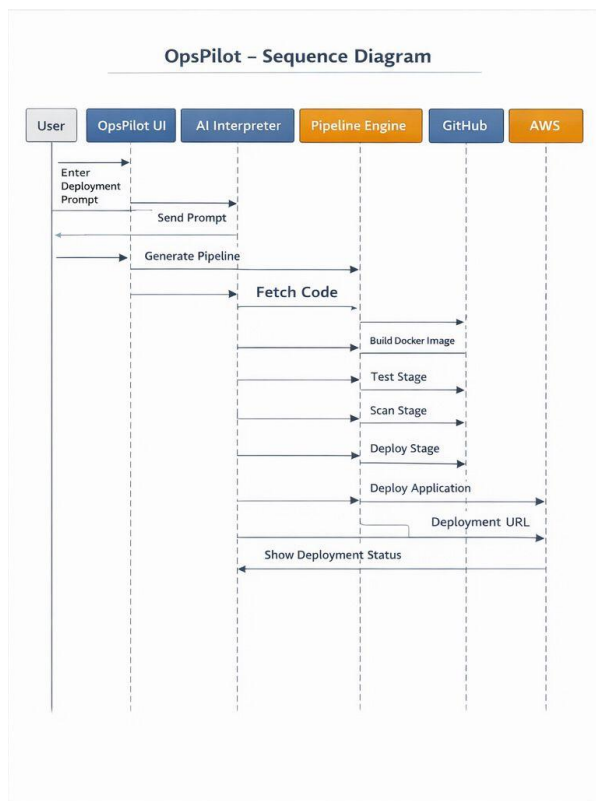
2. DFD level 1:



3. Class Diagram



4. Sequence Diagram



4. Non-Functional Requirements

Non-functional requirements define how well the system performs rather than what it does. For OpsPilot, these requirements are extremely important because the system handles critical operations such as deployments and security scanning.

4.1 Performance Requirements

OpsPilot must be able to process user prompts and generate pipelines quickly. The AI interpretation should occur within a few seconds, and pipeline execution should begin immediately after generation.

The system must support multiple users running pipelines simultaneously without performance degradation. It should efficiently use system resources such as CPU, memory, and network bandwidth. The platform should be scalable so that additional servers can be added as the number of users increases.

4.2 Security Requirements

Security is a core requirement because OpsPilot manages source code, cloud credentials, and deployment environments. All sensitive data such as API keys, passwords, and cloud secrets must be encrypted both in transit and at rest.

The system must implement role-based access control so that users can only access their own projects. Audit logs should be maintained for every action performed. Security scanning tools must be integrated into every pipeline to detect vulnerabilities before deployment.

4.3 Software Quality Attributes

OpsPilot must be reliable, meaning it should not fail during critical operations. It should also be maintainable so that new tools and features can be added easily. Usability is important because the system targets both experts and beginners. Scalability, robustness, and fault tolerance are also essential to ensure long-term success.

5. Other Requirements

OpsPilot must support future expansion and integration with new technologies. The system should allow adding new cloud providers, security tools, and AI models without major redesign. It should also support logging, auditing, and compliance reporting for enterprise use.

The platform should be deployable on both cloud and on-premise environments, making it suitable for organizations with different infrastructure needs.

6. References

The design and implementation of OpsPilot are based on well-established technologies and industry best practices. The main references include Jenkins architecture, GitHub Actions workflows, Docker containerization standards, Kubernetes orchestration principles, and DevSecOps security guidelines.

In addition, the project uses research from Artificial Intelligence and Natural Language Processing to build the prompt interpretation engine. Cloud service documentation from AWS, along with security frameworks such as OWASP and NIST, provide the foundation for secure deployment and monitoring.

These references ensure that OpsPilot is built using reliable, industry-standard approaches.

